

第 3 章

图像运算

针对图像的加法运算、位运算都是比较基础的运算。但是，很多复杂的图像处理功能正是借助这些基础的运算来完成的。所以，牢固掌握基础操作，对于更好地实现图像处理是非常有帮助的。本章简单介绍了加法运算、位运算，并使用它们实现了位平面分解、图像异或加密、数字水印、脸部打码/解码等实例。

3.1 图像加法运算

在图像处理过程中，经常需要对图像进行加法运算。可以通过加号运算符“+”对图像进行加法运算，也可以通过 `cv2.add()` 函数对图像进行加法运算。

通常情况下，在灰度图像中，像素用 8 个比特位（一个字节）来表示，像素值的范围是 $[0, 255]$ 。两个像素值在进行加法运算时，求得的和很可能超过 255。上述两种不同的加法运算方式，对超过 255 的数值的处理方式是不同的。

3.1.1 加号运算符

使用加号运算符“+”对图像 a （像素值为 a ）和图像 b （像素值为 b ）进行求和运算时，遵循以下规则：

$$a + b = \begin{cases} a + b, & a + b \leq 255 \\ \text{mod}(a + b, 256), & a + b > 255 \end{cases}$$

式中，`mod()` 是取模运算，“`mod(a+b, 256)`”表示计算“ $a+b$ 的和除以 256 取余数”。

根据上述规则，两个点进行加法运算时：

- 如果两个图像对应像素值的和小于或等于 255，则直接相加得到运算结果。例如，像素值 28 和像素值 36 相加，得到计算结果 64。
- 如果两个图像对应像素值的和大于 255，则将运算结果对 256 取模。例如 $255+58=313$ ，大于 255，则计算 $(255+58)\% 256 = 57$ ，得到计算结果 57。

当然，上述公式也可以简化为 $a + b = \text{mod}(a + b, 256)$ ，在运算时无论相加的和是否大于 255，都对数值 256 取模。

【例 3.1】 使用随机数数组模拟灰度图像，观察使用“+”对像素值求和的结果。

分析：通过将数组的数值类型定义为 `dtype=np.uint8`，可以保证数组值的范围在 $[0, 255]$ 之间。



根据题目要求及分析，编写程序如下：

```
import numpy as np
img1=np.random.randint(0,256,size=[3,3],dtype=np.uint8)
img2=np.random.randint(0,256,size=[3,3],dtype=np.uint8)
print("img1=\n",img1)
print("img2=\n",img2)
print("img1+img2=\n",img1+img2)
```

运行程序，得到如下计算结果：

```
img1=
[[178  83  29]
 [202 200 158]
 [ 27 177 162]]
img2=
[[ 26  48  57]
 [ 52 153   8]
 [ 10 232   7]]
img1+img2=
[[204 131  86]
 [254  97 166]
 [ 37 153 169]]
```

从上述程序可以看到，使用“+”计算两个 256 级灰度图像内像素值的和时，运算结果会对 256 取模。

需要注意，本例题中的加法要进行取模，这是由数组的类型 `dtype=np.uint8` 所规定的。

3.1.2 cv2.add()函数

函数 `cv2.add()` 可以用来计算图像像素值相加的和，其语法格式为：

计算结果=`cv2.add(像素值 a, 像素值 b)`

使用函数 `cv2.add()` 对像素值 *a* 和像素值 *b* 进行求和运算时，会得到像素值对应图像的饱和值（最大值）。例如，8 位灰度图像的饱和值为 255，因此，在对 8 位灰度图的像素值求和时，遵循以下规则：

$$a + b = \begin{cases} a + b, & a + b \leq 255 \\ 255, & a + b > 255 \end{cases}$$

根据上述规则，在 256 级的灰度图像（8 位灰度图）中的两个像素点进行加法运算时：

- 如果两个像素值的和小于或等于 255，则直接相加得到运算结果。例如，像素值 28 和像素值 36 相加，得到计算结果 64。
- 如果两个像素值的和大于 255，则将运算结果处理为饱和值 255。例如 255+58=313，大于 255，则得到计算结果 255。

需要注意，函数 `cv2.add()` 中的参数可能有如下三种形式。

- 形式 1：计算结果=`cv2.add(图像 1, 图像 2)`，两个参数都是图像，此时参与运算的图像大

小和类型必须保持一致。

- 形式 2: 计算结果=cv2.add(数值,图像), 第 1 个参数是数值, 第 2 个参数是图像, 此时将超过图像饱和值的数值处理为饱和值 (最大值)。
- 形式 3: 计算结果=cv2.add(图像,数值), 第 1 个参数是图像, 第 2 个参数是数值, 此时将超过图像饱和值的数值处理为饱和值 (最大值)。

上述三种形式将在本章的后续小节中进一步介绍。

【例 3.2】 使用随机数组模拟灰度图像, 观察函数 cv2.add() 对像素值求和的结果。

根据题目要求, 编写程序如下:

```
import numpy as np
import cv2
img1=np.random.randint(0,256,size=[3,3],dtype=np.uint8)
img2=np.random.randint(0,256,size=[3,3],dtype=np.uint8)
print("img1=\n",img1)
print("img2=\n",img2)
img3=cv2.add(img1,img2)
print("cv2.add(img1,img2)=\n",img3)
```

运行程序, 得到如下计算结果:

```
img1=
[[136 212  1]
 [ 47 234 85]
 [197 107 169]]
img2=
[[109 212 62]
 [ 19 218 245]
 [ 19 103 137]]
cv2.add(img1,img2)=
[[245 255 63]
 [ 66 255 255]
 [216 210 255]]
```

【例 3.3】 分别使用加号运算符和函数 cv2.add() 计算两幅灰度图像的像素值之和, 观察处理结果。

根据题目要求, 编写程序如下:

```
import cv2
a=cv2.imread("lena.bmp",0)
b=a
result1=a+b
result2=cv2.add(a,b)
cv2.imshow("original",a)
cv2.imshow("result1",result1)
cv2.imshow("result2",result2)
cv2.waitKey()
cv2.destroyAllWindows()
```



在本例中，首先读取了图像 `lena` 并将其标记为变量 `a`；接下来，使用语句“`b=a`”将图像 `lena` 复制到变量 `b` 内；最后，分别使用“+”和函数 `cv2.add()` 计算 `a` 和 `b` 之和。

运行程序，得到如图 3-1 所示的运行结果，其中：

- 左图是原始图像 `lena`。
- 中间的图是使用加号运算符将图像 `lena` 自身相加的结果。
- 右图是使用函数 `cv2.add()` 将图像 `lena` 自身相加的结果。

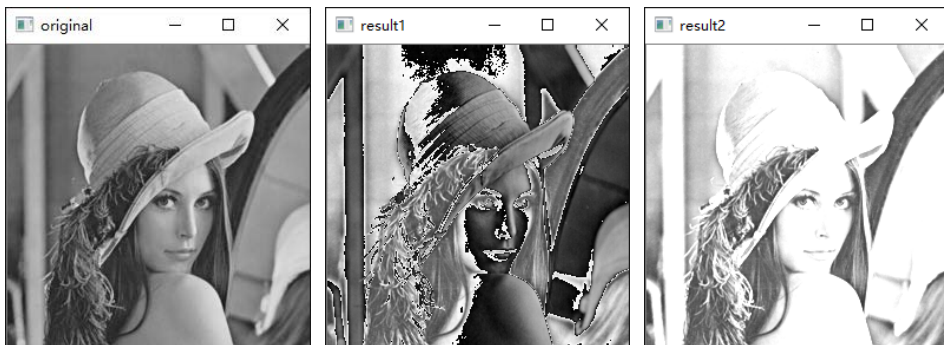


图 3-1 【例 3.3】程序的运行结果

从上述运算结果可以看出：

- 使用加号运算符计算图像像素值的和时，将和大于 255 的值进行了取模处理，取模后大于 255 的这部分值变得更小了，导致本来应该更亮的像素点变得更暗了，相加所得的图像看起来并不自然。
- 使用函数 `cv2.add()` 计算图像像素值的和时，将和大于 255 的值处理为饱和值 255。图像像素值相加后让图像的像素值增大了，图像整体变亮。

3.2 图像加权和

所谓图像加权和，就是在计算两幅图像的像素值之和时，将每幅图像的权重考虑进来，可以用公式表示为：

$$\text{dst} = \text{saturate}(\text{src1} \times \alpha + \text{src2} \times \beta + \gamma)$$

式中，`saturate()` 表示取饱和值（最大值）。图像进行加权和计算时，要求 `src1` 和 `src2` 必须大小、类型相同，但是对具体是什么类型和通道没有特殊限制。它们可以是任意数据类型，也可以有任意数量的通道（灰度图像或者彩色图像），只要二者相同即可。

OpenCV 中提供了函数 `cv2.addWeighted()`，用来实现图像的加权和（混合、融合），该函数的语法格式为：

```
dst=cv2.addWeighted(src1, alpha, src2, beta, gamma)
```

其中，参数 `alpha` 和 `beta` 是 `src1` 和 `src2` 所对应的系数，它们的和可以等于 1，也可以不等于 1。该函数实现的功能是 `dst = src1×alpha + src2×beta + gamma`。需要注意，式中参数 `gamma` 的值可以是 0，但是该参数是必选参数，不能省略。可以将上式理解为“结果图像=图像 1×系数 1+

图像 $2\times$ 系数 2+亮度调节量”。

【例 3.4】 使用数组演示函数 `cv2.addWeighted()` 的使用。

根据题目要求，编写程序如下：

```
1. import cv2
2. import numpy as np
3. img1=np.ones((3,4),dtype=np.uint8)*100
4. img2=np.ones((3,4),dtype=np.uint8)*10
5. gamma=3
6. img3=cv2.addWeighted(img1,0.6,img2,5,gamma)
7. print(img3)
```

本例中的各行代码功能如下。

- 第 3 行生成一个 3×4 大小的、元素数值都是 100 的二维数组，对应一个灰度图像 `img1`。
- 第 4 行生成一个 3×4 大小的、元素数值都是 10 的二维数组，对应一个灰度图像 `img2`。
- 第 5 行将调节亮度参数 `gamma` 的值设置为 3。
- 第 6 行计算 “`img1 $\times$ 0.6+img2 $\times$ 5+3`” 的混合值。

运行程序，得到如下运行结果：

```
[[113 113 113 113]
 [113 113 113 113]
 [113 113 113 113]]
```

【例 3.5】 使用函数 `cv2.addWeighted()` 对两幅图像进行加权混合，观察处理结果。

根据题目要求，编写程序如下：

```
import cv2
a=cv2.imread("boat.bmp")
b=cv2.imread("lena.bmp")
result=cv2.addWeighted(a,0.6,b,0.4,0)
cv2.imshow("boat",a)
cv2.imshow("lena",b)
cv2.imshow("result",result)
cv2.waitKey()
cv2.destroyAllWindows()
```

本程序使用 `cv2.addWeighted()` 函数，对图像 `boat` 和图像 `lena` 分别按照 0.6 和 0.4 的权重进行混合。

运行程序，得到如图 3-2 所示的结果，其中：

- 左图是原始图像 `boat`。
- 中间的图是原始图像 `lena`。
- 右图是图像 `boat` 和图像 `lena` 加权混合后的结果图像。



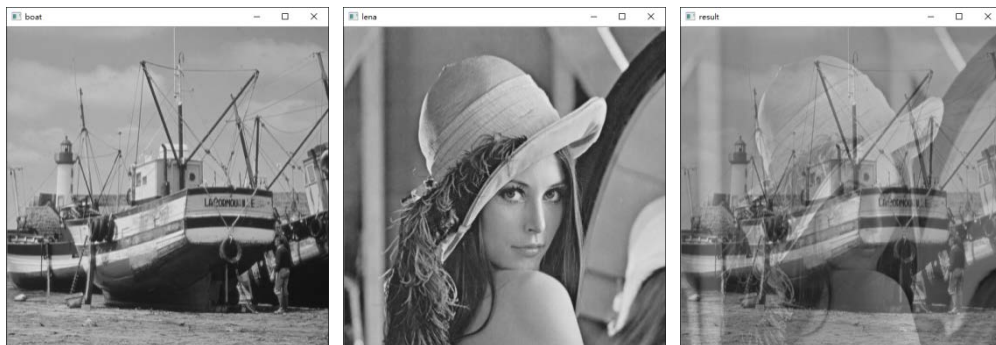


图 3-2 【例 3.5】程序的运行结果

【例 3.6】使用函数 `cv2.addWeighted()` 将一幅图像的 ROI 混合在另外一幅图像内。

根据题目要求，编写程序如下：

```
import cv2
lena=cv2.imread("lena512.bmp",cv2.IMREAD_UNCHANGED)
dollar=cv2.imread("dollar.bmp",cv2.IMREAD_UNCHANGED)
cv2.imshow("lena",lena)
cv2.imshow("dollar",dollar)
face1=lena[220:400,250:350]
face2=dollar[160:340,200:300]
add=cv2.addWeighted(face1,0.6,face2,0.4,0)
dollar[160:340,200:300]=add
cv2.imshow("result",dollar)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，`face1` 是图像 `lena` 中的面部部分，`face2` 是图像 `dollar` 中的面部部分。通过函数 `cv2.addWeighted()` 将 `lena` 图像内的面部 `face1` 与 `dollar` 图像内的面部 `face2` 进行了混合计算。

运行程序，会得到如图 3-3 所示的结果，其中：

- 左图是原始图像 `lena`。
- 中间的图是原始图像 `dollar`。
- 右图是图像 `lena` 的面部与图像 `dollar` 的面部加权混合得到的图像。

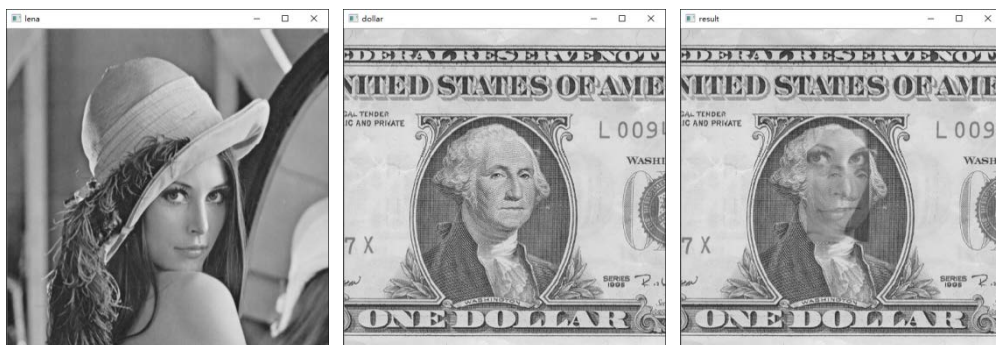


图 3-3 图像加权混合结果

3.3 按位逻辑运算

逻辑运算是一种非常重要的运算方式，图像处理过程中经常要按照位进行逻辑运算，本节介绍 OpenCV 中的按位逻辑运算，简称位运算。

在 OpenCV 内，常见的位运算函数如表 3-1 所示。

表 3-1 常见的位运算函数

函数名	基本含义
cv2.bitwise_and()	按位与
cv2.bitwise_or()	按位或
cv2.bitwise_xor()	按位异或
cv2.bitwise_not()	按位取反

3.3.1 按位与运算

在与运算中，当参与与运算的两个逻辑值都是真时，结果才为真。其逻辑关系可以类比如图 3-4 所示的串联电路，只有当两个开关都闭合时，灯才会亮。

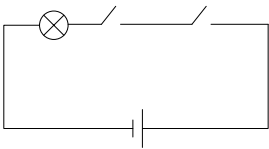


图 3-4 与运算类比电路

表 3-2 对与运算算子的不同情况进行了说明，表中使用“and”表示与运算。

表 3-2 与运算

算子 1	算子 2	结果	规则
0	0	0	and(0,0)=0
0	1	0	and(0,1)=0
1	0	0	and(1,0)=0
1	1	1	and(1,1)=1

按位与运算是指将数值转换为二进制值后，在对应的位置上进行与运算。例如，表 3-3 展示了两个数值进行按位与运算的示例。

表 3-3 按位与运算

数值	十进制值	二进制值
数值 1	198	1100 0110
数值 2	219	1101 1011
按位与运算结果	194	1100 0010

在 OpenCV 中，可以使用 cv2.bitwise_and()函数来实现按位与运算，其语法格式为：

dst = cv2.bitwise_and(src1, src2[, mask]])

式中：

- `dst` 表示与输入值具有同样大小的 `array` 输出值。
- `src1` 表示第一个 `array` 或 `scalar` 类型的输入值。
- `src2` 表示第二个 `array` 或 `scalar` 类型的输入值。
- `mask` 表示可选操作掩码，8 位单通道 `array`。

按位与操作有如下特点：

- 将任何数值 N 与数值 0 进行按位与操作，都会得到数值 0。
- 将任何数值 N （这里仅考虑 8 位值）与数值 255（8 位二进制数是 1111 1111）进行按位与操作，都会得到数值 N 本身。

可以通过表 3-4 观察数值 N （表中是 219）与特殊值 0 和 255 进行按位与运算的结果。

表 3-4 与特殊值 0 和 255 进行按位与运算

按位与运算	二进制值	十进制值	二进制值	十进制值
数值 N	1101 1011	219	1101 1011	219
特殊值（0 及 255）	0000 0000	0	1111 1111	255
运算结果	0000 0000	0	1101 1011	219
说明	数值 219 与数值 0 按位与得到 0		数值 219 与数值 255 按位与，结果保持自身值 219 不变	

根据上述特点，可以构造一幅掩模图像 M ，掩模图像 M 中只有两种值：一种是数值 0，另外一种数值 255。将该掩模图像 M 与一幅灰度图像 G 进行按位与操作，在得到的结果图像 R 中：

- 与掩模图像 M 中的数值 255 对应位置上的值，来源于灰度图像 G 。
- 与掩模图像 M 中的数值 0 对应位置上的值为零（黑色）。

第 13 章将从另外一个角度对上述情况进行说明，以帮助大家更好地理解掩模及处理方式。

【例 3.7】 使用数组演示与掩模图像的按位与运算。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
a=np.random.randint(0,255,(5,5),dtype=np.uint8)
b=np.zeros((5,5),dtype=np.uint8)
b[0:3,0:3]=255
b[4,4]=255
c=cv2.bitwise_and(a,b)
print("a=\n",a)
print("b=\n",b)
print("c=\n",c)
```

运行上述程序，输出结果如下：

```
a=
[[ 39 177 191 66 179]
 [109 28 7 17 118]
 [241 191 32 72 202]
 [229 62 9 65 187]
 [103 65 207 45 121]]
```



```

b=
[[255 255 255  0  0]
 [255 255 255  0  0]
 [255 255 255  0  0]
 [  0  0  0  0  0]
 [  0  0  0  0 255]]
c=
[[ 39 177 191  0  0]
 [109 28  7  0  0]
 [241 191 32  0  0]
 [  0  0  0  0  0]
 [  0  0  0  0 121]]

```

从程序可以看出, 数组 *c* 来源于数组 *a* 与数组 *b* 的按位与操作。运算结果显示, 对于数组 *c* 内的值, 与数组 *b* 中数值 255 对应位置上的值来源于数组 *a*; 与数组 *b* 中数值 0 对应位置上的值为 0。

【例 3.8】 构造一个掩模图像, 使用按位与运算保留图像中被掩模指定的部分。

在本例中, 我们构造一个掩模图像, 保留图像 *lena* 的头部。

根据题目要求, 编写代码如下:

```

import cv2
import numpy as np
a=cv2.imread("lena.bmp",0)
b=np.zeros(a.shape,dtype=np.uint8)
b[100:400,200:400]=255
b[100:500,100:200]=255
c=cv2.bitwise_and(a,b)
cv2.imshow("a",a)
cv2.imshow("b",b)
cv2.imshow("c",c)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序, 输出结果如图 3-5 所示, 左图是原始图像 *lena*, 中间的图是掩模图像, 右图是按位与结果图像, 可以看到, 被掩模指定的头部图像被保留在了运算结果中。

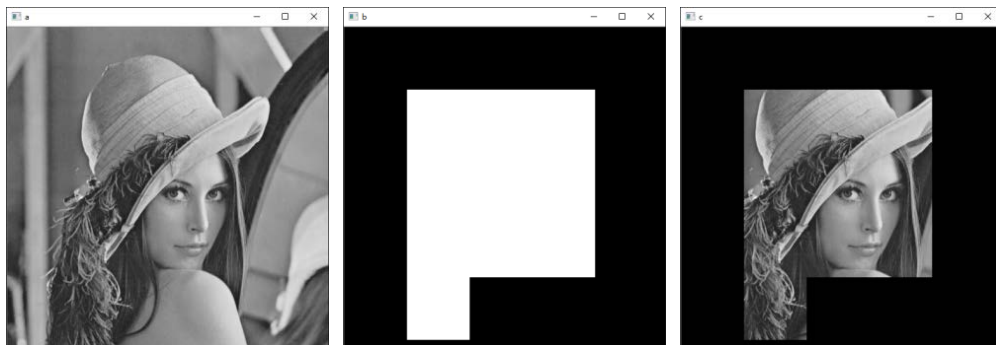


图 3-5 【例 3.8】程序的运行结果

除了需要对灰度图像进行掩模处理，还经常需要针对 BGR 模式的彩色图像使用掩模提取指定部分。由于按位与操作要求参与运算的数据有相同的通道，所以无法直接将彩色图像与单通道的掩模图像进行按位与操作。一般情况下，可以通过将掩模图像转换为 BGR 模式的彩色图像，让彩色图像与掩模图像进行按位与操作，实现掩模运算。

【例 3.9】 构造一个掩模图像，使用按位与操作保留图像内被掩模所指定的部分。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
a=cv2.imread("lena.bmp",1)
b=np.zeros(a.shape,dtype=np.uint8)
b[100:400,200:400]=255
b[100:500,100:200]=255
c=cv2.bitwise_and(a,b)
print("a.shape=",a.shape)
print("b.shape=",b.shape)
cv2.imshow("a",a)
cv2.imshow("b",b)
cv2.imshow("c",c)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，输出结果如图 3-6 所示，其中左图是原始图像，中间的图是掩模图像，右图是原始图像和掩模图像按位与后提取的图像。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

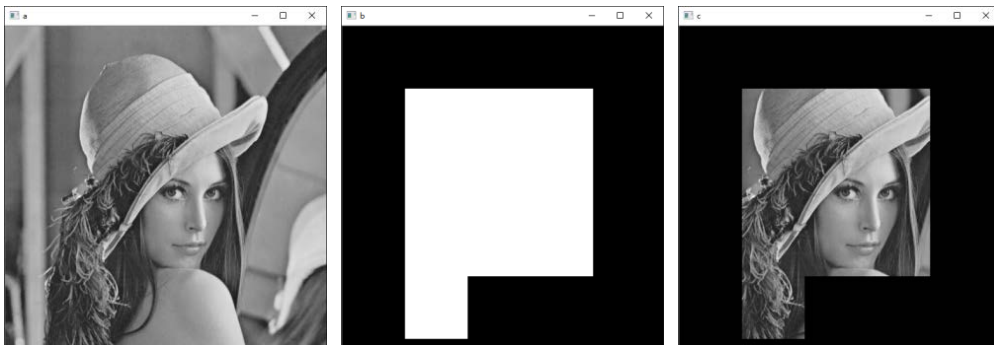


图 3-6 【例 3.9】程序的运行结果

同时，程序还会显示如下结果：

```
a.shape= (512, 512, 3)
b.shape= (512, 512, 3)
```

3.3.2 按位或运算

或运算的规则是，当参与或运算的两个逻辑值中有一个为真时，结果就为真。其逻辑关系可以类比为如图 3-7 所示的并联电路，两个开关中只要有任意一个闭合时，灯就会亮。

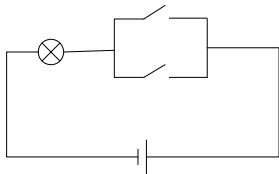


图 3-7 或运算类比电路图

表 3-5 对参与或运算的算子的不同情况进行了说明，表中使用“or”表示或运算。

表 3-5 或运算

算子 1	算子 2	结果	规则
0	0	0	or(0,0)=0
0	1	1	or (0,1)=1
1	0	1	or (1,0)=1
1	1	1	or (1,1)=1

按位或运算是指将数值转换为二进制值后，在对应的位置上进行或运算。例如，表 3-6 展示了两个数值进行按位或运算的示例。

表 3-6 按位或运算

数值	十进制值	二进制值
数值 1	198	1100 0110
数值 2	219	1101 1011
按位或运算结果	223	1101 1111

在 OpenCV 中，可以使用 cv2.bitwise_or()函数来实现按位或运算，其语法格式为：

```
dst = cv2.bitwise_or( src1, src2[, mask]] )
```

式中：

- dst 表示与输入值具有同样大小的 array 输出值。
- src1 表示第一个 array 或 scalar 类型的输入值。
- src2 表示第二个 array 或 scalar 类型的输入值。
- mask 表示可选操作掩码，8 位单通道 array 值。

3.3.3 按位非运算

非运算是取反操作，满足如下逻辑：

- 当运算数为真时，结果为假。
- 当运算数为假时，结果为真。

表 3-7 对参与运算算子的不同情况进行了说明，表中使用“not”表示非运算。

表 3-7 非运算

算子	结果	规则
0	1	not(0)=1
1	0	not(1)=0

按位非运算是指将数值转换为二进制值后，在对应的位置上进行非运算。例如，表 3-8 展示了按位非运算的示例。

表 3-8 按位非运算

说明	十进制值	二进制值
原始数值	198	1100 0110
按位非运算结果	57	0011 1001

在 OpenCV 中，可以使用函数 `cv2.bitwise_not()` 来实现按位取反操作，其语法格式为：

```
dst = cv2.bitwise_not( src[, mask] )
```

式中：

- `dst` 表示与输入值具有同样大小的 `array` 输出值。
- `src` 表示 `array` 类型的输入值。
- `mask` 表示可选操作掩码，8 位单通道 `array` 值。

3.3.4 按位异或运算

异或运算也叫半加运算，其运算法则与不带进位的二进制加法类似，其英文为“exclusive OR”，因此其函数通常表示为 `xor`。

表 3-9 对参与异或运算的算子的不同情况进行了说明，其中“`xor`”表示异或运算。

表 3-9 异或运算

算子 1	算子 2	结果	规则
0	0	0	<code>xor(0,0)=0</code>
0	1	1	<code>xor(0,1)=1</code>
1	0	1	<code>xor(1,0)=1</code>
1	1	0	<code>xor(1,1)=0</code>

按位异或运算是指将数值转换为二进制值后，在对应的位置上进行异或运算。例如，表 3-10 展示了两个数值进行按位异或运算的示例。

表 3-10 按位异或运算

数值	十进制值	二进制值
数值 1	198	1100 0110
数值 2	219	1101 1011
按位异或运算结果	29	0001 1101

在 OpenCV 中，可以使用函数 `cv2.bitwise_xor()` 来实现按位异或运算，其语法格式为：

```
dst = cv2.bitwise_xor( src1, src2[, mask] )
```

式中：

- `dst` 表示与输入值具有同样大小的 `array` 输出值。
- `src1` 表示第一个 `array` 或 `scalar` 类型的输入值。
- `src2` 表示第二个 `array` 或 `scalar` 类型的输入值。
- `mask` 表示可选操作掩码，8 位单通道 `array` 值。

3.4 掩模

OpenCV 中的很多函数都会指定一个掩模，也被称为掩码，例如：

计算结果=cv2.add(参数1, 参数2, 掩模)

当使用掩模参数时，操作只会在掩模值为非空的像素点上执行，并将其他像素点的值置为0。

例如，img1、img2、mask 和 img3 的原始值分别为：

$$\begin{aligned} \text{img1} &= \begin{bmatrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{bmatrix} \\ \text{img2} &= \begin{bmatrix} 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \end{bmatrix} \\ \text{mask} &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\ \text{img3} &= \begin{bmatrix} 66 & 66 & 66 & 66 \\ 66 & 66 & 66 & 66 \\ 66 & 66 & 66 & 66 \\ 66 & 66 & 66 & 66 \end{bmatrix} \end{aligned}$$

经过 $\text{img3}=\text{cv2.add}(\text{img1}, \text{img2}, \text{mask}=\text{mask})$ 运算后，得到 img3 为：

$$\text{img3} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 8 \\ 0 & 0 & 8 & 8 \end{bmatrix}$$

在运算过程中，img3 计算的是在掩模 mask 控制下的“img1+img2”结果。在计算时，掩模为 1 的部分对应“img1+img2”，其他部分的像素值均为“0”。

需要说明的是，在运算前，数组 img3 内就存在值，这仅仅是为了说明问题用的，实际上 img3 是根据函数 cv2.add() 所生成的新数组，与原来的值没有关系。

【例 3.10】 演示掩码的使用。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img1=np.ones((4,4),dtype=np.uint8)*3
img2=np.ones((4,4),dtype=np.uint8)*5
mask=np.zeros((4,4),dtype=np.uint8)
mask[2:4,2:4]=1
img3=np.ones((4,4),dtype=np.uint8)*66
print("img1=\n",img1)
print("img2=\n",img2)
print("mask=\n",mask)
```



```
print("初始值 img3=\n",img3)
img3=cv2.add(img1,img2,mask=mask)
print("求和后 img3=\n",img3)
```

运行上述程序，得到如下输出结果：

```
img1=
[[3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]]
img2=
[[5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]]
mask=
[[0 0 0 0]
 [0 0 0 0]
 [0 0 1 1]
 [0 0 1 1]]
初始值 img3=
[[66 66 66 66]
 [66 66 66 66]
 [66 66 66 66]
 [66 66 66 66]]
求和后 img3=
[[0 0 0 0]
 [0 0 0 0]
 [0 0 8 8]
 [0 0 8 8]]
```

上述例题介绍的是在 `cv2.add()` 函数中使用掩模的情况，在位运算中也都含有掩模参数。在 3.3.1 节中，我们介绍了直接使用按位与运算对原始图像与掩模进行计算的方式。在将彩色图像与掩模进行计算时，由于按位与操作要求参与运算的数据应该有相同的通道，所以无法直接将彩色图像与单通道的掩模图像进行按位与操作。我们通过将掩模图像转换为 BGR 模式的彩色图像，让彩色图像与（彩色）掩模图像进行按位与操作，从而实现掩模运算。

实际上，在函数中所使用的掩模参数可以是 8 位单通道图像。所以，可以将掩模图像作为按位与函数 `cv2.bitwise_and( src1, src2[, mask] )` 中参数 `mask` 的值，完成掩模运算。此时，让待处理的彩色图像同时作为函数 `cv2.bitwise_and( src1, src2[, mask] )` 的参数 `src1` 和参数 `src2`，使用掩模图像作为掩模参数，完成按位与运算，即可得到由掩模控制的彩色图像。

需要注意的是，任何数值与自身进行按位与计算的结果，得到的仍旧是自身的值。例如，在表 3-11 中，数值 198 与自身进行按位与运算，得到的结果仍旧是 198。

表 3-11 数值与自身按位与运算示例

说明	十进制值	二进制值
数值	198	1100 0110
	198	1101 1011
按位与运算结果	198	1101 1011

所以，在上述操作中，让待处理的彩色图像与自身进行按位与操作，得到的仍是彩色图像本身。而使用的掩模参数控制的是，在目标图像中，哪些区域的值是彩色图像的值、哪些区域的值是 0。

【例 3.11】 构造一个掩模图像，将该掩模图像作为按位与函数的掩模参数，实现保留图像的指定部分。

```
import cv2
import numpy as np
a=cv2.imread("lena.bmp",1)
w,h,c=a.shape
mask=np.zeros((w,h),dtype=np.uint8)
mask[100:400,200:400]=255
mask[100:500,100:200]=255
c=cv2.bitwise_and(a,a,mask)
print("a.shape=",a.shape)
print("mask.shape=",mask.shape)
cv2.imshow("a",a)
cv2.imshow("mask",mask)
cv2.imshow("c",c)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，输出结果如图 3-8 所示，其中左图为原始图像，中间的图为掩模图像，右图为原始图像与掩模图像进行按位与后的图像。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

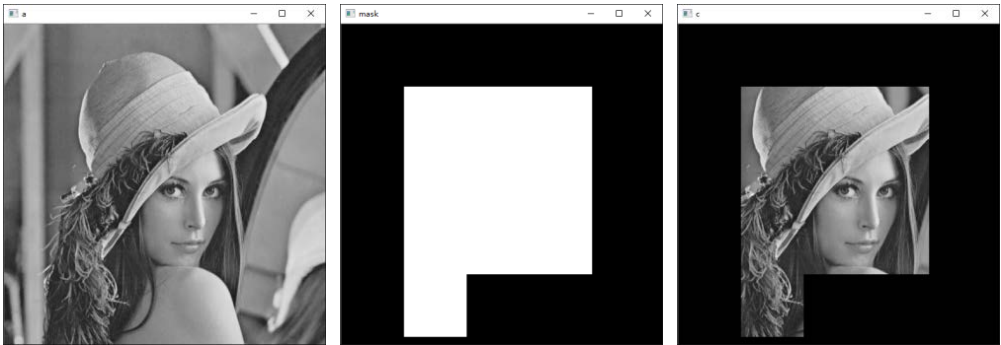


图 3-8 彩色图像按位与提取

除此以外，程序还将显示如下结果：

```
a.shape= (512, 512, 3)
mask.shape= (512, 512)
```


3.5 图像与数值的运算

在上述加法运算和按位运算中，参与运算的两个算子（参数）既可以是两幅图像，也可以是一幅图像与一个数值。

例如，如果想增加图像的整体亮度，可以将每一个像素值都加上一个特定值。在具体实现时，可以给图像加上一个统一像素值的图像，也可以给图像加上一个固定值。

例如，img1 和 img2 的原始值分别为：

$$\text{img1} = \begin{bmatrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{bmatrix}$$

$$\text{img2} = \begin{bmatrix} 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \end{bmatrix}$$

经过 `img3=cv2.add(img1,img2)` 运算后，得到 img3 为：

$$\text{img3} = \begin{bmatrix} 8 & 8 & 8 & 8 \\ 8 & 8 & 8 & 8 \\ 8 & 8 & 8 & 8 \\ 8 & 8 & 8 & 8 \end{bmatrix}$$

经过 `img4=cv2.add(img1,6)` 运算后，得到 img4 为：

$$\text{img4} = \begin{bmatrix} 9 & 9 & 9 & 9 \\ 9 & 9 & 9 & 9 \\ 9 & 9 & 9 & 9 \\ 9 & 9 & 9 & 9 \end{bmatrix}$$

经过 `img5=cv2.add(6,img2)` 运算后，得到 img5 为：

$$\text{img5} = \begin{bmatrix} 11 & 11 & 11 & 11 \\ 11 & 11 & 11 & 11 \\ 11 & 11 & 11 & 11 \\ 11 & 11 & 11 & 11 \end{bmatrix}$$

【例 3.12】 演示图像与数值的运算结果。

为了方便理解，本例中采用数组模拟图像演示运算结果。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img1=np.ones((4,4),dtype=np.uint8)*3
img2=np.ones((4,4),dtype=np.uint8)*5
print("img1=\n",img1)
print("img2=\n",img2)
img3=cv2.add(img1,img2)
print("cv2.add(img1,img2)=\n",img3)
img4=cv2.add(img1,6)
print("cv2.add(img1,6)\n",img4)
```

```
img5=cv2.add(6,img2)
print("cv2.add(6,img2)=\n",img5)
```

运行上述程序，得到如下输出结果：

```
img1=
[[3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]]
img2=
[[5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]
 [5 5 5 5]]
cv2.add(img1,img2)=
[[8 8 8 8]
 [8 8 8 8]
 [8 8 8 8]
 [8 8 8 8]]
cv2.add(img1,6)
[[9 9 9 9]
 [9 9 9 9]
 [9 9 9 9]
 [9 9 9 9]]
cv2.add(6,img2)=
[[11 11 11 11]
 [11 11 11 11]
 [11 11 11 11]
 [11 11 11 11]]
```

3.6 位平面分解

将灰度图像中处于同一比特位上的二进制像素值进行组合，得到一幅二进制值图像，该图像被称为灰度图像的一个位平面，这个过程被称为位平面分解。例如，将一幅灰度图像内所有像素点上处于二进制位内最低位上的值进行组合，可以构成“最低有效位”位平面。

在8位灰度图中，每一个像素使用8位二进制值来表示，其值的范围在[0,255]之间。可以将其中的值表示为：

$$\text{value} = a_7 \times 2^7 + a_6 \times 2^6 + a_5 \times 2^5 + a_4 \times 2^4 + a_3 \times 2^3 + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0$$

式中， a_i 的可能值为0或1。可以看出，各个 a_i 的权重是不一样的， a_7 的权重最高， a_0 的权重最低。这代表 a_7 的值对图像的影响最大，而 a_0 的值对图像的影响最小。

通过提取灰度图像像素点二进制像素值的每一比特位的组合，可以得到多个位平面图像。图像中全部像素值的 a_i 值所构成的位平面，称为第 i 个位平面（第 i 层）。在8位灰度图中，可以组成8个二进制值图像，即可以将原图分解为8个位平面。



根据上述分析，像素值中各个 a_i 的权重是不一样的：

- a_7 的权重最高，所构成的位平面与原图像相关性最高，该位平面看起来通常与原图像最类似。
- a_0 权重最低，所构成的位平面与原图像相关性最低，该平面看起来通常是杂乱无章的。

借助按位与运算可以实现位平面分解。例如，有灰度图像 O 的像素值为：

209	197	163	193
125	247	160	112
161	137	243	203
39	82	154	127

其对应的二进制值为：

1101 0001	1100 0101	1010 0011	1100 0001
0111 1101	1111 0111	1010 0000	0111 0000
1010 0001	1000 1001	1111 0011	1100 1011
0010 0111	0101 0010	1001 1010	0111 1111

将所有像素的 a_i 值进行组合，便会得到图像的 8 个位平面，如图 3-9 所示。

1 1 1 1	0 0 1 0	0 1 0 0	0 0 0 0
1 1 0 0	0 1 0 0	1 1 0 0	1 0 0 0
1 1 1 1	0 0 1 1	0 0 0 0	0 1 0 1
1 0 0 1	1 1 1 1	1 0 0 1	0 0 1 1
(a)	(b)	(c)	(d)
1 0 0 0	0 0 1 0	1 1 0 1	1 1 1 1
1 1 0 1	1 1 1 1	1 1 0 1	0 1 1 0
0 0 1 0	1 0 1 0	0 0 1 1	1 1 1 1
0 1 1 1	1 0 0 1	0 1 0 1	0 0 1 0
(e)	(f)	(g)	(h)

图 3-9 位平面分解

图 3-9 中的各个位平面的构成如下。

- 图(a)是由图像 O 中每个像素的 a_0 值（即从右边数第 0 个二进制位，第 0 个比特位）组成的，我们称之为第 0 个位平面，也可以称为第 0 层，即是由二进制位的第 0 位构成的。为了叙述和理解上的方便，其序号从 0 开始，也可以称为“最低有效位”位平面。
- 图(b)是由图像 O 中每个像素的 a_1 值（即从右数第 1 个比特位）组成的，我们称之为第 1 个位平面（或第 1 层）。
- 图(c)是由图像 O 中每个像素的 a_2 值（即从右数第 2 个比特位）组成的，我们称之为第 2 个位平面（或第 2 层）。
- 图(d)是由图像 O 中每个像素的 a_3 值（即从右数第 3 个比特位）组成的，我们称之为第 3 个位平面（或第 3 层）。
- 图(e)是由图像 O 中每个像素的 a_4 值（即从右数第 4 个比特位）组成的，我们称之为第 4 个位平面（或第 4 层）。

- 图(f)是由图像 O 中每个像素的 a_5 值（即从右数第 5 个比特位）组成的，我们称之为第 5 个位平面（或第 5 层）。
- 图(g)是由图像 O 中每个像素的 a_6 值（即从右数第 6 个比特位）组成的，我们称之为第 6 个位平面（或第 6 层）。
- 图(h)是由图像 O 中每个像素的 a_7 值（即从右数第 7 个比特位）组成的，我们称之为第 7 个位平面（或第 7 层），也可以称为“最高有效位”位平面。

针对 RGB 图像，如果将 R 通道、G 通道、B 通道中的每一个通道对应的位平面进行合并，即可组成新的 RGB 彩色图像。例如，针对一幅 RGB 图像，将其 R 通道的第 3 个位平面、G 通道的第 3 个位平面、B 通道的第 3 个位平面进行合并，则可以构成一幅新的 RGB 彩色图像，我们称之为原始图像的第 3 个位平面。通过上述方式，可以完成彩色图像的位平面分解。

下面以灰度图像为例，介绍位平面分解的具体步骤。

1. 图像预处理

读取原始图像 O，获取原始图像 O 的宽度 M 和高度 N。

2. 构造提取矩阵

使用按位与操作能够很方便地将一个数值指定位上的数字提取出来。例如，在表 3-12 中，分别使用不同的提取因子 F 来提取数值 N 中的特定位。可以发现，提取因子 F 哪位上的值为 1，就可以提取数值 N 中哪位上的数字。

表 3-12 位值提取示例

说明	第 0 位	第 3 位	第 5 位	第 7 位
数值 N (其中 X 值为 0 或 1)	XXXX XXXX	XXXX XXXX	XXXX XXXX	XXXX XXXX
提取因子 F	0000 0001	0000 1000	0001 0000	1000 0000
提取结果	0000 000X	0000 X000	000X 0000	X 000 0000

根据上述分析结果，建立一个值均为 2^n 的 Mat 作为提取矩阵（数组），用来与原始图像进行按位与运算，以提取第 n 个位平面。

提取矩阵 Mat 内可能的值如表 3-13 所示。

表 3-13 Mat 内可能的值

要提取的 位平面序号	Mat 值 计算方法	Mat 内部值	Mat 值的二进制表示
0	2^0	1	0000 0001
1	2^1	2	0000 0010
2	2^2	4	0000 0100
3	2^3	8	0000 1000
4	2^4	16	0001 0000
5	2^5	32	0010 0000
6	2^6	64	0100 0000
7	2^7	128	1000 0000



3. 提取位平面

将灰度图像与提取矩阵进行按位与运算，得到各个位平面。

将像素值与一个值为 2^n 的数值进行按位与运算，能够使像素值的第 n 位保持不变，而将其余各位均置零。因此，通过像素值与特定值的按位与运算，能够提取像素值的指定二进制位的值。同理，通过按位与运算，能够提取图像的指定位平面。

例如，有一个像素点的像素值为 219，要提取其第 4 个二进制位的值，即提取该像素值的第 4 位信息（序号从 0 开始）。此时，需要借助的提取值是 “ $2^4=16$ ”，具体的运算如表 3-14 所示。

表 3-14 提取像素值示例

运算	类别	十进制值	二进制值
按位与	像素值	219	1101 1011
	提取值	$2^4=16$	0001 0000
结果		16	0001 0000

从表 3-14 可以看到，通过将像素值 219 与提取值 2^4 的二进制值进行按位与运算，提取到了像素值 219 第 4 位上的二进制数字 “1”。这是因为在 2^4 的二进制表示中，只有第 4 位上的数字为 1，其余各位的数字都是 0。在像素值 219 与提取值 2^4 的二进制值进行按位与运算时：

- 像素值 219 第 4 位上的数字 “1” 与提取值 2^4 第 4 位上的数字 “1” 进行与操作，数字 “1” 被保留。
- 像素值 219 其他位上的数字与提取值 2^4 其他位上的数字 “0” 进行与操作，所以都会变为 0。

在针对图像的位平面提取中，例如有图像 O，其像素值为：

209	196	163	193
125	247	160	114
161	9	227	201
39	86	154	127

其对应的二进制形式标记为 RB，具体如下：

1101 0001	1100 0100	1010 0011	1100 0001
0111 1101	1111 0111	1010 0000	0111 0010
1010 0001	0000 1001	1110 0011	1100 1001
0010 0111	0101 0110	1001 1010	0111 1111

如果想提取其中的第 3 个位平面，则需要建立元素值均为 2^3 的数组 BD，其中的值全部为 8（即 2^3 ），具体如下：

8	8	8	8
8	8	8	8
8	8	8	8
8	8	8	8

将数组 BD 所对应的二进制形式标记为 BT，具体为：



0000 1000	0000 1000	0000 1000	0000 1000
0000 1000	0000 1000	0000 1000	0000 1000
0000 1000	0000 1000	0000 1000	0000 1000
0000 1000	0000 1000	0000 1000	0000 1000

将 RB 与 BT 进行按位与运算，得到 RE，具体为：

0000 0000	0000 0000	0000 0000	0000 0000
0000 1000	0000 0000	0000 0000	0000 0000
0000 0000	0000 1000	0000 0000	0000 1000
0000 0000	0000 0000	0000 1000	0000 1000

将 RE 所对应的十进制形式标记为 RD，具体为：

0	0	0	0
8	0	0	0
0	8	0	8
0	0	8	8

【提示】提取位平面也可以通过将二进制像素值右移指定位，然后对 2 取模得到。例如，要提取第 n 个位平面，则可以将像素向右侧移动 n 位，然后对 2 取模，就可以得到第 n 个位平面。

4. 阈值处理

通过计算得到的位平面是一个二值图像，如果直接将上述得到的位平面显示出来，则会得到一张近似黑色的图像。这是因为默认当前显示的图像是 8 位灰度图，而当其中的像素值较小时，显示的图像就会是近似黑色的。例如，在图像 RD 中，最大的像素值是 8，因此几乎为纯黑色。要想让 8 显示为白色，必须将 8 处理为 255。

也就是说，每次提取位平面后，要想让二值位平面能够以黑白颜色显示出来，就要将得到的二值位平面进行阈值处理，将其中大于零的值处理为 255。

例如，将得到的位平面 RD 进行阈值处理，将其中的 8 调整为 255，具体语句为：

```
mask=RD[:, :, i]>0
RD[mask]=255
```

首先，使用 `mask=RD[:, :, i]>0` 对 RD 进行处理：

- 将 RD 中大于 0 的值处理为逻辑值真（True）。
- 将 RD 中小于或等于 0 的值处理为逻辑值假（False）。

按照上述处理，得到的 mask 为：

False	False	False	False
True	False	False	False
False	True	False	True
False	False	True	True

接下来，使用 `RD[mask]=255`，将 RD 中对应“mask 中逻辑值为真”位置上的值替换为 255。在阈值调整后的位平面 RD 中，原来为 8 的值被替换为 255，具体如下：

0	0	0	0
255	0	0	0
0	255	0	255
0	0	255	255

需要说明的是，在 OpenCV 中，提供了专门的函数用来进行阈值处理。关于阈值处理的相关内容会在第 6 章进行详细介绍。

5. 显示图像

完成上述处理后，可以将位平面显示出来，直观地观察各个位平面的具体情况。

【例 3.13】 编写程序，观察灰度图像的各个位平面。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
lena=cv2.imread("lena.bmp",0)
cv2.imshow("lena", lena)
r,c=lena.shape
x=np.zeros((r,c,8),dtype=np.uint8)
for i in range(8):
    x[:, :, i]=2**i
r=np.zeros((r,c,8),dtype=np.uint8)
for i in range(8):
    r[:, :, i]=cv2.bitwise_and( lena,x[:, :, i])
    mask=r[:, :, i]>0
    r[mask]=255
    cv2.imshow(str(i),r[:, :, i])
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，通过两个循环提取了灰度图像的各个位平面，具体说明如下。

- 使用 `x=np.zeros((r,c,8),dtype=np.uint8)` 语句设置一个用于提取各个位平面的提取矩阵。该矩阵是“`rxc×8`”大小的，其中 `r` 是行高，`c` 是列宽，`8` 表示共有 8 个通道。`r`、`c` 的值来源于要提取的图像的行高、列宽。矩阵 `x` 的 8 个通道分别用来提取灰度图像的 8 个位平面。例如，`x[:, :, 0]` 用来提取灰度图像的第 0 个位平面。
- 在第 1 个 `for` 循环中，使用 `x[:, :, i]=2**i` 语句设置用于提取各个位平面的提取矩阵的值。
- 在第 2 个 `for` 循环中，实现了各个位平面的提取、阈值处理和显示。

运行上述程序，得到如图 3-10 所示的图像，其中：

- 图(a)是原始 `lena` 图像。
- 图(b)是第 0 个位平面，第 0 个位平面位于 8 位二进制值的最低位，其权重最低，对像素值的影响最小，与 `lena` 图像的相关度也最低，所以显示出来的是一幅杂乱无章的图像。
- 图(c)是第 1 个位平面。

- 图(d)是第2个位平面。
- 图(e)是第3个位平面。
- 图(f)是第4个位平面。
- 图(g)是第5个位平面。
- 图(h)是第6个位平面。
- 图(i)是第7个位平面，第7个位平面位于8位二进制值的最高位，其对像素值的影响最大。第7位二进制值在8位二进制数中权重最高，与lena图像的相关度最高。所以，第7个位平面是与原始图像最接近的二值图像。

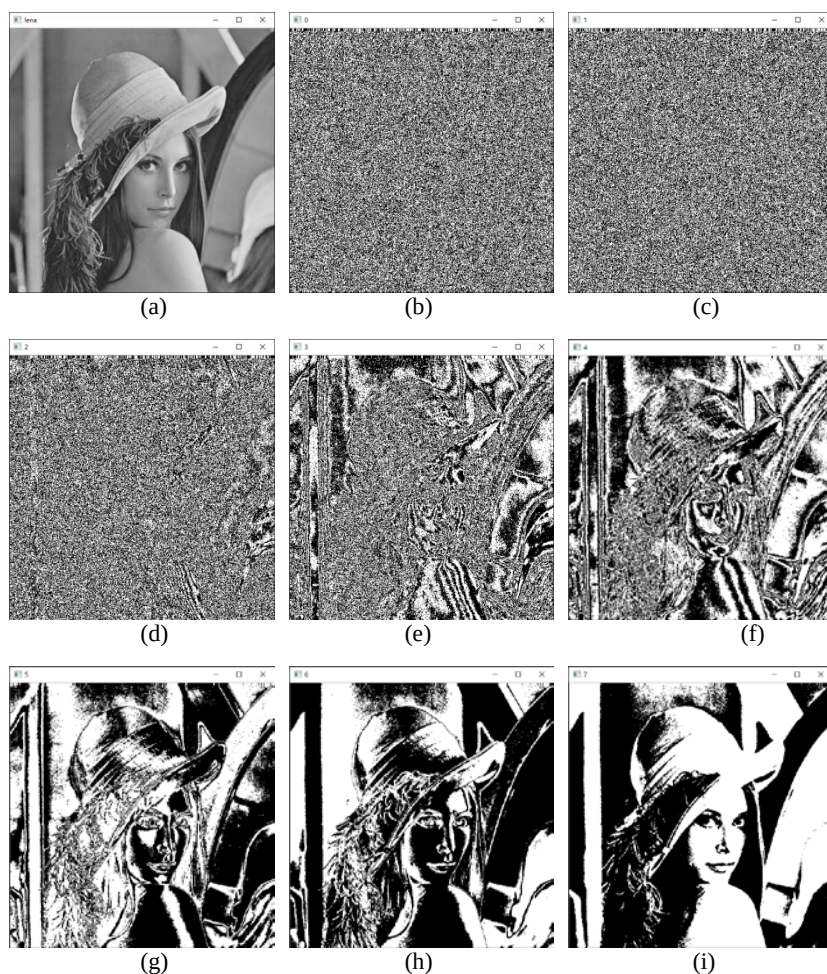


图 3-10 位平面分解效果图

3.7 图像加密和解密

通过按位异或运算可以实现图像的加密和解密。

通过对原始图像与密钥图像进行按位异或，可以实现加密；将加密后的图像与密钥图像再

次进行按位异或，可以实现解密。

按位异或运算的基本规则如表 3-15 所示。

表 3-15 按位异或运算的基本规则

算子 1	算子 2	结果	规则
0	0	0	xor(0,0)=0
0	1	1	xor(0,1)=1
1	0	1	xor(1,0)=1
1	1	0	xor(1,1)=0

根据上述按位异或运算的规则，假设：

`xor(a,b)=c`

则可以得到：

`xor(c,b)=a`

`xor(c,a)=b`

上述运算的过程如表 3-16 所示。

表 3-16 按位异或运算的过程示例

a	b	c (xor(a,b))	xor(c,b) (=a)	xor(c,a) (=b)
0	0	0	0	0
0	1	1	0	1
1	0	1	1	0
1	1	0	1	1

从上述结果可以看出，如果上述 a、b、c 具有如下关系：

- a：明文，原始数据。
- b：密钥。
- c：密文，通过 xor(a,b)实现。

则可以对上述数据进行如下操作和理解。

- 加密过程：将明文 a 与密钥 b 进行按位异或，完成加密，得到密文 c。
- 解密过程：将密文 c 与密钥 b 进行按位异或，完成解密，得到明文 a。

位运算是针对二进制位进行的运算，利用位运算即可实现对像素点的加密。在图像处理中，需要处理的像素点的值通常为灰度值，其范围通常为[0,255]。例如，某个像素点的值为 216（明文），则可以使用 178（该数值由加密者自由选定）作为密钥对其进行加密，让这两个数的二进制值进行按位异或运算，即完成加密，得到一个密文 106。当需要解密时，将密文 106 与密钥 178 进行按位异或运算，即可得到原始像素点值 216（明文）。具体过程为：

`bit_xor(216,178)=106`

`bit_xor(106,178)=216`

以二进制形式表示的具体细节如下。

• 加密过程

运算	说明	二进制值	十进制值
bit_xor	明文	1101 1000	216
	密钥	1011 0010	178
运算结果	密文	0110 1010	106

• 解密过程

运算	说明	二进制值	十进制值
bit_xor	密文	0110 1010	106
	密钥	1011 0010	178
运算结果	明文	1101 1000	216

对图像内的每一个像素点重复上述操作，即可完成对图像的加密、解密操作。这里以一个原始图像 O 为例，具体说明图像的加密、解密过程。

1. 加密过程

假设有需要加密的原始图像 O，其中的像素值为：

202	120	30	156
249	25	23	190
238	1	198	150
6	21	65	24

选定的加密密钥图像为 K，其中的像素值为：

15	212	223	71
183	6	30	235
218	18	157	172
4	178	95	154

图像 O 所对应的二进制表示 OB 为：

1100 1010	0111 1000	0001 1110	1001 1100
1111 1001	0001 1001	0001 0111	1011 1110
1110 1110	0000 0001	1100 0110	1001 0110
0000 0110	0001 0101	0100 0001	0001 1000

密钥图像 K 所对应的二进制表示 KB 为：

0000 1111	1101 0100	1101 1111	0100 0111
1011 0111	0000 0110	0001 1110	1110 1011
1101 1010	0001 0010	1001 1101	1010 1100
0000 0100	1011 0010	0101 1111	1001 1010

将 OB 与 KB 进行按位异或运算，即得到图像 O 的加密图像 OSB：

1100 0101	1010 1100	1100 0001	1101 1011
0100 1110	0001 1111	0000 1001	0101 0101
0011 0100	0001 0011	0101 1011	0011 1010
0000 0010	1010 0111	0001 1110	1000 0010

将 OSB 转换为十进制形式 OS，如下：



197	172	193	219
78	31	9	85
52	19	91	58
2	167	30	130

至此，图像 O 的加密过程完成，得到原始图像 O 的加密图像 OS。

2. 解密过程

解密过程需要将加密图像 OS 与密钥图像 K 进行按位异或运算，得到原图像 OR。

将加密图像 OS 的二进制形式 OSB 与密钥图像 K 的二进制形式 KB 进行按位异或运算，即得到原始图像 OR 的二进制形式 ORB。按照上述运算，得到的 ORB 为：

1100 1010	0111 1000	0001 1110	1001 1100
1111 1001	0001 1001	0001 0111	1011 1110
1110 1110	0000 0001	1100 0110	1001 0110
0000 0110	0001 0101	0100 0001	0001 1000

将 ORB 转换为十进制形式，得到解密图像 OR，如下：

202	120	30	156
249	25	23	190
238	1	198	150
6	21	65	24

至此，图像的解密过程结束，得到加密图像 OS 的解密图像 OR。

从上述过程可以看到，解密过程所得到的解密图像 OR 与原始图像 O 是一致的。这说明上述加密、解密过程是正确的。

上述说明过程中，为了方便理解和观察数据的运算，在进行按位运算时，我们都是将十进制数转换为二进制数后，再进行位运算处理的。实际上，在使用 OpenCV 编写程序时，不需要这样转换，OpenCV 中位运算函数的参数是十进制数，位运算函数会直接对十进制参数进行按位异或运算。

【例 3.14】编写程序，通过图像按位异或实现加密和解密过程。

在具体实现中，甲乙双方可以通过协商预先确定一幅密钥图像，并且双方各保存一份备用。在此基础上，甲乙双方就可以利用该密钥图像进行图像的加密和解密处理了。例如，甲通过密钥对原始图像加密后，原始图像变得杂乱无章，其他人无法解读图像内容。而乙可以通过预先保存的密钥图像，将加密图像解密，获取原始图像内容。

在加密过程中，可以选择一幅有意义的图像作为密钥，也可以选择一幅没有意义的图像作为密钥。在本例中，将随机生成一幅图像作为密钥。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
lena=cv2.imread("lena.bmp",0)
r,c=lena.shape
key=np.random.randint(0,256,size=[r,c],dtype=np.uint8)
```

```

encryption=cv2.bitwise_xor(lena,key)
decryption=cv2.bitwise_xor(encryption,key)
cv2.imshow("lena",lena)
cv2.imshow("key",key)
cv2.imshow("encryption",encryption)
cv2.imshow("decryption",decryption)
cv2.waitKey()
cv2.destroyAllWindows()

```

本例的各个图像关系如下。

- 图像 lena 是明文（原始）图像，是需要加密的图像，从当前目录下读入。
- 图像 key 是密钥图像，是加密和解密过程中所使用的密钥，该图像是由随机数生成的。
- 图像 encryption 是加密图像，是明文图像 lena 和密钥图像 key 通过按位异或运算得到的。
- 图像 decryption 是解密图像，是加密图像 encryption 和密钥图像 key 通过按位异或运算得到的。

运行上述程序，结果如图 3-11 所示，其中：

- 图(a)是原始图像 lena。
- 图(b)是密钥图像 key。
- 图(c)是原始图像 lena（图(a)）借助密钥 key（图(b)）加密得到的加密图像 encryption。
- 图(d)是对加密图像 encryption（图(c)）使用密钥图像 key（图(b)）解密得到的解密图像 decryption。

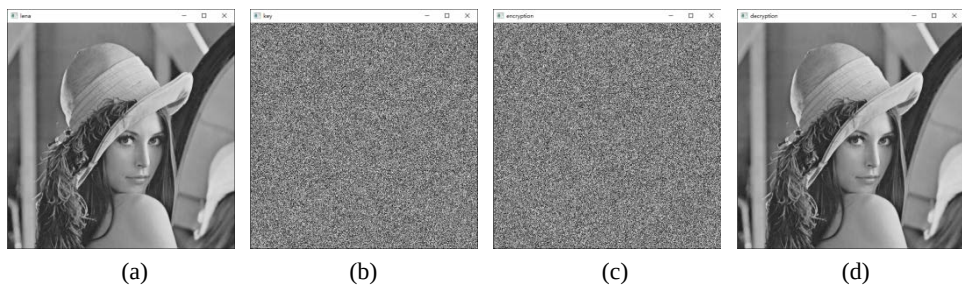


图 3-11 【例 3.14】程序的运行结果

3.8 数字水印

最低有效位（Least Significant Bit, LSB）指的是一个二进制数中的第 0 位（即最低位）。最低有效位信息隐藏指的是，将一个需要隐藏的二值图像信息嵌入载体图像的最低有效位，即将载体图像的最低有效位层替换为当前需要隐藏的二值图像，从而实现将二值图像隐藏的目的。由于二值图像处于载体图像的最低有效位上，所以对于载体图像的影响非常不明显，其具有较高的隐蔽性。

在必要时直接将载体图像的最低有效位层提取出来，即可得到嵌入在该位上的二值图像，达到提取秘密信息的目的。



这种信息隐藏也被称为数字水印，通过该方式可以实现信息隐藏、版权认证、身份认证等功能。例如，如果嵌入载体图像内的信息是秘密信息，就实现了信息隐藏；如果嵌入载体图像内的信息是版权信息，就能够实现版权认证；如果嵌入载体图像内的信息是身份信息，就可以实现数字签名，等等。所以，被嵌入载体图像内的信息也被称为数字水印信息。

数字水印信息可以是文本、视频、音频等多种形式，这里我们仅讨论数字水印信息是二值图像的情况。

3.8.1 原理

从位平面的角度考虑，数字水印的处理过程分为下面两步。

- 嵌入过程：将载体图像的第 0 个位平面替换为数字水印信息（一幅二值图像）。
- 提取过程：将载体图像的最低有效位所构成的第 0 个位平面提取出来，得到数字水印信息。

1. 嵌入过程

嵌入过程是将数字水印嵌入载体图像的过程。该过程实现的是，将载体图像的最低有效位用数字水印信息替换，得到包含水印信息的载体图像。

为了处理和说明起来方便，这里以原始图像为灰度图像、水印图像为二值图像为例。在实际处理中，原始图像、水印图像均可以为彩色图像，这时需要先对它们进行通道分解、图层分解，后续的处理方法与在灰度图像内嵌入二值水印图像的处理方法相同。

（1）原始载体图像预处理

为了便于理解，我们将载体图像处理为二进制形式，并标记出最低有效位。

例如，有灰度图像 O 作为载体图像，图像 O 中各个像素点的像素值为：

209	197	163	193
125	247	160	112
161	137	243	203
39	82	154	127

其对应的二进制表示 OB 为：

1101 000 <u>1</u>	1100 010 <u>1</u>	1010 001 <u>1</u>	1100 000 <u>1</u>
0111 110 <u>1</u>	1111 011 <u>1</u>	1010 000 <u>0</u>	0111 000 <u>0</u>
1010 000 <u>1</u>	1000 100 <u>1</u>	1111 001 <u>1</u>	1100 101 <u>1</u>
0010 011 <u>1</u>	0101 001 <u>0</u>	1001 101 <u>0</u>	0111 111 <u>1</u>

为了更直观，我们用下划线将图像的二进制表示 OB 的最低有效位标记出来了。被标记的值构成了载体图像 O 的第 0 个位平面，即“最低有效位”位平面 OBLSB，其具体值为：

1	1	1	1
1	1	0	0
1	1	1	1
1	0	0	1



(2) 水印图像处理

为了方便处理，在嵌入水印前，需要将水印信息处理为二值图像。比较典型的情况是将灰度二值水印信息进行阈值处理，将其处理为二进制二值水印信息。

在灰度二值图像中，像素值只有 0 和 255 两种类型值，分别用来表示黑色和白色。可以将其中的 255 转换为 1，这样就得到了一幅二进制二值图像。在二进制二值图像中，仅仅用一个比特位表示一个像素值，像素值只有 0 和 1 两种可能值。经过上述处理，能够更方便地实现水印嵌入。

例如，有灰度二值水印图像 **W**，其像素值为：

255	0	255	0
255	255	0	255
0	0	0	0
255	255	0	255

不能直接将上述水印图像嵌入载体图像内，需要将其转换为二进制二值水印图像，以方便嵌入。通过阈值处理，得到二进制二值水印图像 **WB**，其具体值为：

1	0	1	0
1	1	0	1
0	0	0	0
1	1	0	1

经过处理后，二进制二值水印信息正好可以嵌入载体图像的最低有效位上。

(3) 嵌入水印

将载体图像的最低有效位替换为二进制水印图像，完成水印的嵌入。

例如，将载体图像 **OB** 的最低有效位用水印信息 **WB** 替换，得到含水印载体图像的二进制形式 **WOB**，其具体值为：

1101 000 <u>1</u>	1100 010 <u>0</u>	1010 001 <u>1</u>	1100 000 <u>0</u>
0111 110 <u>1</u>	1111 011 <u>1</u>	1010 000 <u>0</u>	0111 000 <u>1</u>
1010 0000	1000 100 <u>0</u>	1111 0010	1100 1010
0010 011 <u>1</u>	0101 001 <u>1</u>	1001 101 <u>0</u>	0111 111 <u>1</u>

将 **WOB** 转为十进制形式，即得到含水印载体图像的十进制值形式 **WO**，其值为：

209	196	163	192
125	247	160	113
160	136	242	202
39	83	154	127

由于信息的最低有效位对值的大小影响有限，因此，将载体图像最低有效位的值用水印信息替换后，载体图像像素的值并没有发生太大变化，人眼不足以看出区别，水印具有较高的隐蔽性。



2. 提取过程

提取过程是指将水印信息从包含水印信息的载体图像内提取出来的过程。提取水印时，先将含水印载体图像的像素值转换为二进制形式，然后从其最低有效位提取出水印信息即可。因此，可以通过提取含水印载体图像的“最低有效位”位平面的方式来得到水印信息。

例如，有包含水印信息的载体图像 WOE，其具体值为：

209	196	163	192
125	247	160	113
160	136	242	202
39	83	154	127

将含水印信息的载体图像 WOE 转换为二进制形式 WOEB，得到：

1101 000 <u>1</u>	1100 010 <u>0</u>	1010 001 <u>1</u>	1100 000 <u>0</u>
0111 110 <u>1</u>	1111 011 <u>1</u>	1010 000 <u>0</u>	0111 000 <u>1</u>
1010 000 <u>0</u>	1000 100 <u>0</u>	1111 001 <u>0</u>	1100 101 <u>0</u>
0010 011 <u>1</u>	0101 001 <u>1</u>	1001 101 <u>0</u>	0111 111 <u>1</u>

提取 WOEB 的最低有效位信息（“最低有效位”位平面，即第 0 个位平面），即可得到水印信息 WE，其值为：

1	0	1	0
1	1	0	1
0	0	0	0
1	1	0	1

根据需要，决定是否进行阈值处理。如有必要，则通过阈值处理将其中值为 1 的像素点转换为 255，得到含有 0 和 255 两个值的二值水印图像 WET，具体值为：

255	0	255	0
255	255	0	255
0	0	0	0
255	255	0	255

通过上述例题可以发现，经过上述处理后，得到的水印图像 WET 与嵌入的水印图像 W 是一致的。

为了便于理解，这里仅介绍了原始载体图像为灰度图像的情况，在实际中可以根据需要在多个通道内嵌入相同的水印（提高鲁棒性，即使部分水印丢失，也能提取出完整水印信息），或在各个不同的通道内嵌入不同的水印（提高嵌入容量）。在彩色图像的多个通道内嵌入水印的方法，与在灰度图像内嵌入水印的方法相同。

3.8.2 实现方法

最低有效位水印的实现包含嵌入过程和提取过程，下面对具体的实现方法进行简单介绍。

1. 嵌入过程

嵌入过程完成的操作是，将数字水印信息嵌入载体图像内，其主要步骤如下。



(1) 载体图像预处理

读取原始载体图像，并获取载体图像的行数 M 和列数 N 。

例如，有原始载体图像 O ，其像素值为：

179	177	136	83
207	87	226	227
222	117	11	84
85	231	52	189

将其对应的二进制形式记为 OB ，其具体值为：

1011 0011	1011 0001	1000 1000	0101 0011
1100 1111	0101 0111	1110 0010	1110 0011
1101 1110	0111 0101	0000 1011	0101 0100
0101 0101	1110 0111	0011 0100	1011 1101

(2) 建立提取矩阵

建立一个 $M \times N$ 大小、元素值均为 254 的提取矩阵（数组），用来提取载体图像的高七位。

例如，按照原始图像 O 的大小建立一个 4×4 大小、元素值均为 254 的数组 T ，其具体为：

254	254	254	254
254	254	254	254
254	254	254	254
254	254	254	254

T 所对应的二进制形式记为 TB ，其具体值为：

1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110

(3) 保留载体图像的高七位，将最低位置零

为了实现该操作，需要将载体图像与元素值均为 254 的提取矩阵进行按位与运算。

将一个值在 $[0,255]$ 之间的像素值 P 与数值 254 进行按位与运算，则会将像素值 P 的最低有效位置零，只保留其高七位。例如：

- 某个像素 P_a 的像素值为 217，将像素 P_a 与 254 进行按位与运算，则像素 P_a 的二进制像素值高七位保持不变，最低有效位被清零。像素 P_a 的最低有效位上原来的值是 1，因此，经过运算后像素 P_a 的像素值 217 减少 1，变为 216。
- 某个像素 P_b 的像素值为 216，将像素 P_b 与 254 进行按位与运算，则像素 P_b 的二进制像素值高七位保持不变，最低有效位被清零。像素 P_b 的最低有效位上原来的值就是 0，因此在运算后像素 P_b 的像素值仍然是 216。

该运算示例具体如表 3-17 所示。



表 3-17 按位与运算示例

运算	运算数	像素 P 的 LSB 为 1		像素 P 的 LSB 为 0	
		二进制值	十进制值	二进制值	十进制值
bit_and	像素 P 的值	1101 1001	217	1101 1000	216
	数值 254	1111 1110	254	1111 1110	254
运算结果		1101 1000	216	1101 1000	216

表中的 LSB 表示 Least Significant Bit，即最低有效位。

根据以上分析，将载体图像与元素值均为 254 的提取矩阵进行按位与运算，相当于将载体图像内的每个像素值均与值 254 进行按位与运算。这样就实现了将整个图像内所有像素二进制值的高七位保留、最低位置零。

例如，将原始载体图像 OB 与元素值均为 254 的提取矩阵 TB 进行按位与运算，则 OB 的高七位保持不变，而最低有效位被置零。即实现了只保留 OB 的高七位，得到 OBH，其具体值为：

1011 001 <u>0</u>	1011 000 <u>0</u>	1000 100 <u>0</u>	0101 001 <u>0</u>
1100 111 <u>0</u>	0101 011 <u>0</u>	1110 001 <u>0</u>	1110 001 <u>0</u>
1101 111 <u>0</u>	0111 010 <u>0</u>	0000 101 <u>0</u>	0101 010 <u>0</u>
0101 010 <u>0</u>	1110 011 <u>0</u>	0011 010 <u>0</u>	1011 110 <u>0</u>

【提示】要保留图像的高七位，还有一种实现方法，即先对像素进行右移一位的位操作，再进行左移一位的位操作。

(4) 水印图像处理

有些情况下需要对水印进行简单处理。例如，当水印图像为 8 位灰度图的二值图像时，就需要将其转换为二进制二值图像，以方便将其嵌入载体图像的最低位。

例如，有一幅灰度二值水印图像 W，具体值为：

255	255	255	255
255	255	255	255
255	0	255	0
0	0	255	255

我们将其中的像素值 255 转换为像素值 1，以方便嵌入载体图像。该灰度二值图像对应的二进制图像为 WT，其值具体为：

1	1	1	1
1	1	1	1
1	0	1	0
0	0	1	1

其对应的 8 位二进制形式 WTB 为：

0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>
0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>
0000 000 <u>1</u>	0000 000 <u>0</u>	0000 000 <u>1</u>	0000 000 <u>0</u>
0000 000 <u>0</u>	0000 000 <u>0</u>	0000 000 <u>1</u>	0000 000 <u>1</u>

(5) 嵌入水印

将原始载体图像进行“保留高七位、最低位置零”的操作后，我们得到一幅新的图像，将新图像与水印图像进行按位或运算，就能实现将水印信息嵌入原始载体图像内的效果。

将一个最低有效位（LSB）为 0 的数值 A 与一个只有一位的二进制值 B（单位二进制值）进行按位或运算时：

- 当该二进制值 B 为 0 时，按位或运算的结果是 0，数值 A 的值保持不变。由于 B 的值为 0，因此，如果从最低有效位的角度理解，可以理解为数值 A 的最低有效位被替换为单位二进制值 B 的值，也可以理解为将单位二进制值 B 嵌入数值 A 内（的最低有效位上）。
- 当该二进制值 B 为 1 时，按位或运算的结果是 1，数值 A 的高七位保持变，而最低有效位变为 1。因此，如果从最低有效位的角度理解，可以理解为数值 A 的最低有效位被替换为单位二进制值 B 的值，也可以理解为将单位二进制值 B 嵌入数值 A 内（的最低有效位上）。

总结来看，将一个最低有效位（LSB）为 0 的数值 A 与一个单位二进制值 B 进行按位或运算，相当于用该单位二进制值 B 替换原始数值 A 的最低有效位，即可以实现将单位二进制值 B 嵌入数值 A 的最低有效位上。

例如，将最低有效位是 0 的数字 216 分别与单位二进制值 0 和单位二进制值 1 进行按位或运算，如表 3-18 所示。

表 3-18 按位或运算示例

运算值	二进制值 B 为 1 即数值 A 与数值 1 进行按位或运算		二进制值 B 为 0 即数值 A 与数值 0 进行按位或运算	
	二进制值	十进制值	二进制值	十进制值
数值 A	1101 1000	216	1101 1000	216
单位二进制值 B	0000 0001	1	0000 0000	0
运算结果	1101 1001	217	1101 1000	216

可以推断，如果将二进制二值水印图像（单位二进制值的水印图像）与最低有效位被置零后得到的原始载体图像进行按位或运算，就可以实现将水印信息嵌入原始载体图像内。

因此，将水印信息 WB 与原始载体图像 O 的高七位图像 OBH 进行按位或运算，即完成将水印信息嵌入原始载体图像 O 的 OBH 内，就可以得到含水印载体图像 WO。WO 的具体值为：

1011 001 <u>1</u>	1011 000 <u>1</u>	1000 100 <u>1</u>	0101 001 <u>1</u>
1100 111 <u>1</u>	0101 011 <u>1</u>	1110 001 <u>1</u>	1110 001 <u>1</u>
1101 111 <u>1</u>	0111 010 <u>0</u>	0000 101 <u>1</u>	0101 010 <u>0</u>
0101 010 <u>0</u>	1110 011 <u>0</u>	0011 010 <u>1</u>	1011 110 <u>1</u>

(6) 显示图像

完成上述处理后，分别显示原始载体图像、水印图像、含水印图像。
水印嵌入过程的流程图如图 3-12 所示。

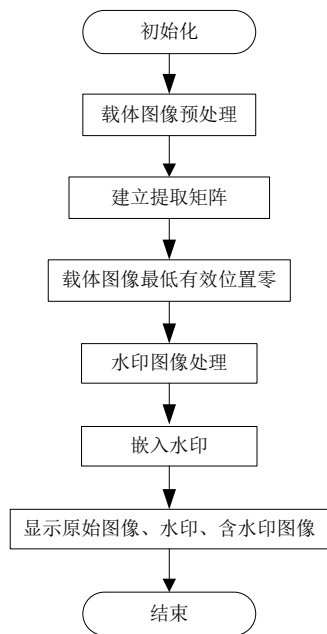


图 3-12 水印嵌入过程流程图

2. 提取过程

提取过程将完成数字水印的提取，具体步骤如下。

(1) 含水印载体图像处理

读取包含水印的载体图像，获取含水印载体图像的大小 $M \times N$ 。

例如，读取含水印载体图像 WO ，获取其大小为 4×4 。 WO 的具体值为：

1011 001 <u>1</u>	1011 000 <u>1</u>	1000 100 <u>1</u>	0101 001 <u>1</u>
1100 111 <u>1</u>	0101 011 <u>1</u>	1110 001 <u>1</u>	1110 001 <u>1</u>
1101 111 <u>1</u>	0111 010 <u>0</u>	0000 101 <u>1</u>	0101 010 <u>0</u>
0101 010 <u>0</u>	1110 011 <u>0</u>	0011 010 <u>1</u>	1011 110 <u>1</u>

(2) 建立提取矩阵

定义一个与含水印载体图像等大小的值为 1 的矩阵（数组）作为提取矩阵。

例如，定义一个大小为 4×4 的矩阵作为提取矩阵，使其中的值均为 1，将该矩阵标记为 $T1$ ：

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

其对应的 8 位二进制形式 Te 为：

0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>
0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>
0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>
0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>	0000 000 <u>1</u>

(3) 提取水印信息

将含水印载体图像与提取矩阵进行按位与运算，提取水印信息。

将一个值在[0, 255]之间的像素 P 与数值 1 进行按位与运算，则会将像素 P 的像素值的高七位置零，只保留像素 P 的最低有效位 (LSB)。

下面分别以像素 P 的最低有效位为 0 和 1 为例进行说明。

- 如果像素 P 的最低有效位为 1，则会得到值 1。
例如，某像素 Pa 的值为 217，将其与数值 1 进行按位与运算，则 Pa 的高七位被置零，只有其最低有效位被保留，得到值 1。
- 如果像素 P 的最低有效位为 0，则会得到值 0。
例如，某像素 Pb 的值为 216，将其与数值 1 进行按位与运算，则 Pb 的高七位被置零，只有其最低有效位被保留，得到值 0。

该实例的具体计算如表 3-19 所示。

表 3-19 与数值 1 进行按位与运算示例

运算	说明	像素 P 的 LSB 为 1		像素 P 的 LSB 为 0	
		二进制值	十进制值	二进制值	十进制值
bit_and	像素 P 的值	1101 1001	217	1101 1000	216
	数值 1	0000 0001	1	0000 0001	1
运算结果		0000 0001	1	0000 0000	0

基于上述规则，针对图像内的每个像素，将其与数值 1 进行按位与操作，即可将图像的最低有效位提取出来。

因此，可以将含水印载体图像与元素值均为 1 的提取矩阵进行按位与运算，以实现提取水印信息。

例如，含水印载体图像 WO 为：

1011 001 <u>1</u>	1011 000 <u>1</u>	1000 100 <u>1</u>	0101 001 <u>1</u>
1100 111 <u>1</u>	0101 011 <u>1</u>	1110 001 <u>1</u>	1110 001 <u>1</u>
1101 111 <u>1</u>	0111 010 <u>0</u>	0000 101 <u>1</u>	0101 010 <u>0</u>
0101 010 <u>0</u>	1110 011 <u>0</u>	0011 010 <u>1</u>	1011 110 <u>1</u>

将含水印图像 WO 与提取矩阵 Te 进行按位与运算，即可得到二进制值水印信息 We，其值为：

1	1	1	1
1	1	1	1
1	0	1	0
0	0	1	1

将提取出来的二进制水印信息 We 进行阈值处理，将其中值为 1 的像素值调整为 255，以便显示。阈值处理后，得到二值水印图像 WG，具体值为：

255	255	255	255
255	255	255	255
255	0	255	0
0	0	255	255



(4) 计算去除水印后的载体图像

有时需要删除包含在水印载体图像内的水印信息。通过将含水印载体图像的最低有效位置零，即可实现删除水印信息。

建立一个大小为 4×4 、元素值均为 254 的矩阵，将该矩阵标记为 T2，其具体为：

254	254	254	254
254	254	254	254
254	254	254	254
254	254	254	254

将上述 T2 所对应的二进制形式记为 TB，其具体值为：

1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110
1111 1110	1111 1110	1111 1110	1111 1110

通过将含水印载体图像 WO 与 TB 进行按位与运算，即可将载体图像 WO 的最低有效位置零，得到删除水印信息的载体图像 ODW。该操作的具体实现原理及过程，与水印嵌入时对原始图像的最低有效位置零操作是类似的。

(5) 显示图像

根据需要，分别显示提取出来的水印图像 WG、删除水印信息的载体图像 ODW。

水印提取过程的流程图如图 3-13 所示。

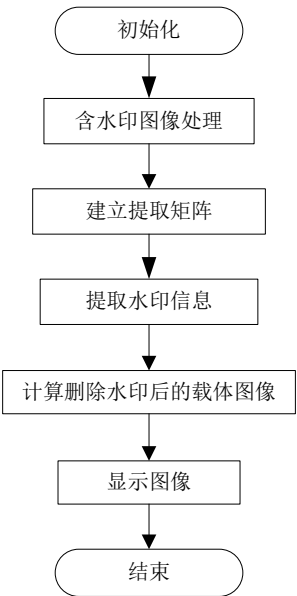


图 3-13 水印提取过程流程图

【提示】将像素值对 2 取模，可以获取像素值的最低有效位。因此，可以通过让含水印载体图像对 2 取模的方式，获取图像的“最低有效位”位平面，提取到的位平面即为水印信息。

就是说,也可以通过让含水印载体图像像素值对2取模的方式,来获取最低有效位水印。

3.8.3 例题

【例 3.15】编写程序,模拟数字水印的嵌入和提取过程。

根据题目要求,编写代码如下:

```
import cv2
import numpy as np
#读取原始载体图像
lena=cv2.imread("lena.bmp",0)
#读取水印图像
watermark=cv2.imread("watermark.bmp",0)
#将水印图像内的值 255 处理为 1, 以方便嵌入
#后续章节会介绍使用 threshold 处理
w=watermark[:, :]>0
watermark[w]=1
#读取原始载体图像的 shape 值
r,c=lena.shape
#=====嵌入过程=====
#生成元素值都是 254 的数组
t254=np.ones((r,c),dtype=np.uint8)*254
#获取 lena 图像的高七位
lenaH7=cv2.bitwise_and(lena,t254)
#将 watermark 嵌入 lenaH7 内
e=cv2.bitwise_or(lenaH7,watermark)
#=====提取过程=====
#生成元素值都是 1 的数组
t1=np.ones((r,c),dtype=np.uint8)
#从载体图像内提取水印图像
wm=cv2.bitwise_and(e,t1)
print(wm)
#将水印图像内的值 1 处理为 255, 以方便显示
#后续章节会介绍使用 threshold 实现
w=wm[:, :]>0
wm[w]=255
#=====显示=====
cv2.imshow("lena", lena)
cv2.imshow("watermark", watermark*255) #当前 watermark 内最大值为 1
cv2.imshow("e", e)
cv2.imshow("wm", wm)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序,结果如图 3-15 所示,其中:

- 图(a)是原始图像 lena。



- 图(b)是水印图像 watermark。在程序中，该图像首先会被处理为二值图像，在显示时将其元素值乘以 255，以方便显示。
- 图(c)是在图像 lena 内嵌入水印图像 watermark 后得到的含水印载体图像 e。
- 图(d)是从含水印载体图像 e 内提取到的水印图像 wm。



图 3-14 【例 3.15】程序的运行结果

从图 3-14 可以发现，通过肉眼无法观察出含水印载体图像和原始图像的不同，水印的隐蔽性较高。但是，由于该方法过于简单，其安全性并不高，在实际处理中会通过更复杂的方式实现水印的嵌入。

3.9 脸部打码及解码

为了帮助读者加深对本章知识点的理解，本节介绍一个使用掩模和按位运算方式实现的对脸部打码、解码实例。

【例 3.16】编写程序，使用掩码对 lena 图像的脸部进行打码、解码。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
#读取原始载体图像
lena=cv2.imread("lena.bmp",0)
#读取原始载体图像的 shape 值
r,c=lena.shape
mask=np.zeros((r,c),dtype=np.uint8)
mask[220:400,250:350]=1
#获取一个 key, 打码、解码所使用的密钥
key=np.random.randint(0,256,size=[r,c],dtype=np.uint8)
#=====获取打码脸=====
#使用密钥 key 对原始图像 lena 加密
lenaXorKey=cv2.bitwise_xor(lena,key)
#获取加密图像的脸部信息 encryptFace
encryptFace=cv2.bitwise_and(lenaXorKey,mask*255)
#将图像 lena 内的脸部值设置为 0，得到 noFace1
noFace1=cv2.bitwise_and(lena,(1-mask)*255)
#得到打码的 lena 图像
maskFace=encryptFace+noFace1
```

```

#=====将打码脸解码=====
#将脸部打码的 lena 与密钥 key 进行异或运算，得到脸部的原始信息
extractOriginal=cv2.bitwise_xor(maskFace,key)
#将解码的脸部信息 extractOriginal 提取出来，得到 extractFace
extractFace=cv2.bitwise_and(extractOriginal,mask*255)
#从脸部打码的 lena 内提取没有脸部信息的 lena 图像，得到 noFace2
noFace2=cv2.bitwise_and(maskFace,(1-mask)*255)
#得到解码的 lena 图像
extractLena=noFace2+extractFace
#=====显示图像=====
cv2.imshow("lena",lena)
cv2.imshow("mask",mask*255)
cv2.imshow("1-mask",(1-mask)*255)
cv2.imshow("key",key)
cv2.imshow("lenaXorKey",lenaXorKey)
cv2.imshow("encryptFace",encryptFace)
cv2.imshow("noFace1",noFace1)
cv2.imshow("maskFace",maskFace)
cv2.imshow("extractOriginal",extractOriginal)
cv2.imshow("extractFace",extractFace)
cv2.imshow("noFace2",noFace2)
cv2.imshow("extractLena",extractLena)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行上述程序，会出现如图 3-15 所示的图像，其中：

- 图(a)是原始图像 lena，本程序要对其脸部进行打码。
- 图(b)是模板图像 mask，其中白色区域的像素值为 1，黑色区域的像素值为 0。为了方便显示，在使用函数 cv2.imshow()显示该图像时，将其中的值 1 调整为 255。
- 图(c)是模板图像 mask（图(b)）的反色图。
- 图(d)是密钥图像 key，该图像使用随机数生成。
- 图(e)是整体打码图像 lenaXorKey，是将图像 lena（图(a)）和密钥图像 key（图(d)）进行异或运算得到的。
- 图(f)是从整体打码图像（图(e)）内提取的脸部打码图像 encryptFace。
- 图(g)是从图像 lena（图(a)）内提取的不包含脸部信息的图像 noFace1，在提取过程中，将模板图像 mask 的反色图（图(c)）作为模板。
- 图(h)是对图像 lena 的脸部进行打码的结果图像 maskFace，该图像是通过将脸部打码图像 encryptFace（图(f)）和不包含脸部信息的图像 noFace1（图(g)）进行按位或运算得到的。
- 图(i)是提取的初步原始图像 extractOriginal，该图像是通过将打码脸部图像 maskFace（图(h)）和密钥图像 key（图(d)）进行异或运算得到的。
- 图(j)是从提取的初步原始图像 extractOriginal（图(i)）中提取的脸部图像 extractFace。
- 图(k)从脸部打码的结果图像 maskFace（图(h)）内提取的不包含脸部信息的图像 noFace2。



- 图(l)是最终的脸部解码结果图像 `extractLena`，该图像是通过提取的脸部图像 `extractFace`（图(j)）和不包含脸部信息的图像 `noFace2`（图(k)）进行按位或运算得到的。

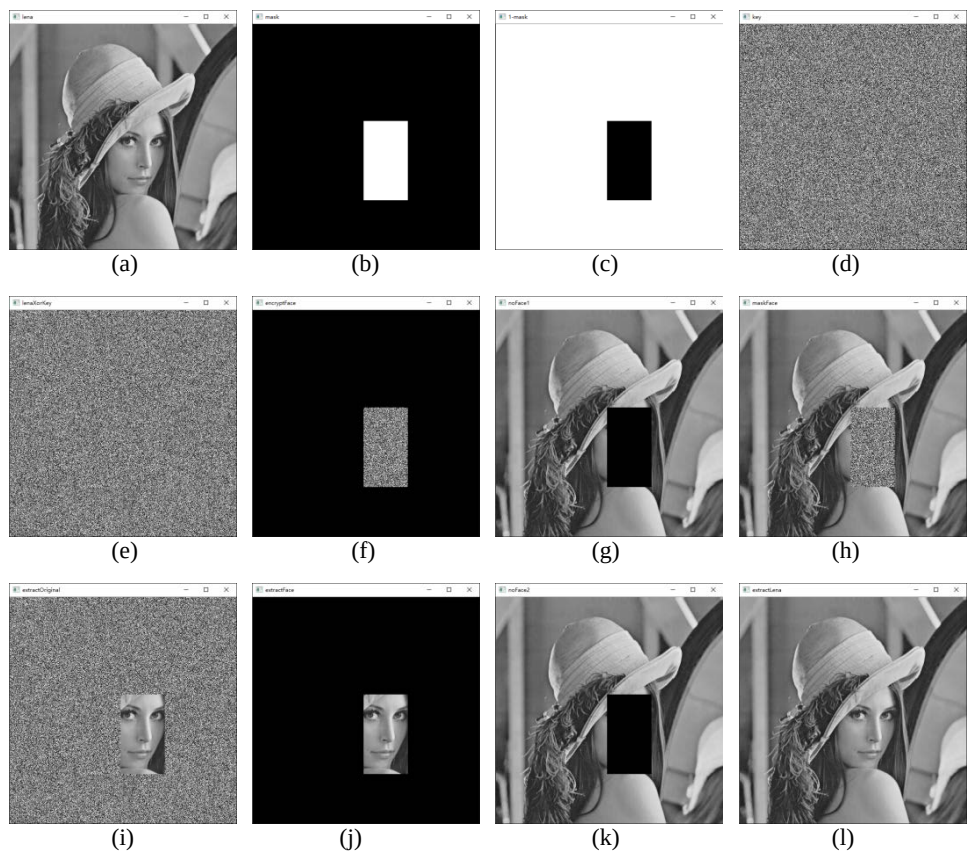


图 3-15 【例 3.16】程序的运行结果